Java + Generative AI Developer

12-Week Detailed Learning & Job-Preparation Roadmap

For an experienced Java developer | 1–2 hours/day | Beginner-level Python

Goal	Build enterprise-style GenAI applications using Java and Spring Boot.
Core AI skills	LLMs, prompting, Spring AI, embeddings, RAG, vector search, tool calling and agent workflows.
Engineering skills	Security, evaluation, testing, observability, resilience, Docker and cloud basics.
Portfolio outcome	A document Q&A assistant, controlled tool-calling workflow and polished demo.

How to use this plan

- Study 60–90 minutes most days; use extra time for coding, debugging or revision.
- Build while learning—do not postpone projects until Week 12.
- Start with one model provider, Spring AI and one vector-store option; avoid tool-hopping.
- Use approved accounts and non-sensitive sample data. Never commit API keys.
- This is a practical target, not a guarantee of job readiness; adjust pace based on progress.

Recommended starting stack

Java + Spring Boot + Maven + Spring AI + one hosted LLM API + PostgreSQL/pgvector + JUnit 5/Mockito + Docker. Learn enough Python for scripts and AI tooling; keep Java as your main backend language.

12-week overview

Week	Focus	Expected output
Week 1	Python Essentials + AI Fundamentals	a small Python LLM script with configuration and error handling.
Week 2	Prompt Engineering + Model APIs	summarisation prototype with tests and README.
Week 3	Spring AI + Spring Boot	Java REST service integrated with an LLM.
Week 4	Embeddings, Vector Search + RAG Foundations	working document retrieval component.
Week 5	Build an Enterprise Document Q&A; Assistant	end-to-end RAG document assistant.
Week 6	RAG Evaluation, Quality + Reliability	reproducible RAG evaluation suite.
Week 7	Tool Calling + Java Service Integration	controlled tool-calling assistant.
Week 8	Agentic Workflows + Orchestration	bounded, observable multi-step workflow.
Week 9	Enterprise Security + Data Protection	documented, tested security controls.
Week 10	Testing, Observability + Production Readiness	testable, observable and resilient service.
Week 11	Docker, CI/CD + Cloud Deployment	containerised app with build/test/deployment instructions.
Week 12	Portfolio, Interviews + Applications	portfolio project, interview notes and targeted application plan.

Daily study rhythm (about 90 minutes)

20 min	Learn one focused concept.
40 min	Write and run code.
20 min	Answer interview questions aloud.
10 min	Review notes, errors and next steps.

Week 1 — Python Essentials + AI Fundamentals

Weekly goal: Build enough Python and AI literacy to understand model-powered applications.

Weekly deliverable: a small Python LLM script with configuration and error handling.

Day 1 — Python setup and basics

- Install Python 3 and create a virtual environment.
- Practise variables, strings, numbers, booleans, type conversion and functions.

Practice/output: Write a script that accepts input and formats a response.

Day 2 — Collections and control flow

- Learn lists, tuples, sets, dictionaries, loops, conditions and comprehensions.
- Practise common collection operations and safe dictionary access.

Practice/output: Count word frequencies from a text file.

Day 3 — Functions, modules and files

- Learn imports, modules, exceptions, logging, and reading/writing text and JSON.
- Understand try/except and basic input validation.

Practice/output: Create a module that reads JSON and validates required fields.

Day 4 — HTTP and REST APIs

- Learn HTTP methods, status codes, headers and JSON payloads.
- Call an API; handle timeouts/errors; use environment variables for secrets.

Practice/output: Write a small API client with error handling.

Day 5 — LLM fundamentals

- Learn tokens, context windows, prompts, inference and model limitations.
- Understand hallucinations, grounding, training vs inference and temperature.

Practice/output: Explain the lifecycle of a model request in your own words.

Day 6 — First model API call

- Choose one approved model provider and follow its current quickstart.
- Send a prompt; inspect responses; add configuration and timeout handling.

Practice/output: Build a command-line text summariser.

Day 7 — Review and mini assessment

- Repeat the API call without copying the tutorial.
- Review Python basics and AI vocabulary; record errors and lessons.

Practice/output: Deliverable: a small Python LLM script with configuration and error handling.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 2 — Prompt Engineering + Model APIs

Weekly goal: Make model calls more predictable, structured and robust.

Weekly deliverable: summarisation prototype with tests and README.

Day 1 — Prompt structure

- Separate instructions, user input and contextual data.
- Use clear constraints, examples, output requirements and task decomposition.

Practice/output: Create prompts to summarise technical incident reports.

Day 2 — Structured output

- Request JSON/schema-constrained output when supported.
- Validate fields, types and missing values in application code.

Practice/output: Return validated JSON with summary, category and action items.

Day 3 — Parameters and context

- Understand token limits, temperature, context windows and output length.
- Compare prompt variants on a fixed set of examples.

Practice/output: Create a small prompt evaluation sheet.

Day 4 — API resilience

- Handle rate limits, HTTP errors, timeouts and transient failures.
- Learn bounded retries and exponential backoff; avoid infinite retries.

Practice/output: Add safe timeout and retry behaviour to your model client.

Day 5 — Cost and latency

- Understand input/output token usage and pricing concepts.
- Measure latency and usage; consider caching with privacy safeguards.

Practice/output: Record latency/usage for five requests.

Day 6 — Prompt injection and data handling

- Treat user input and retrieved content as untrusted.
- Never expose secrets; do not use prompts as the only security boundary.

Practice/output: Test prompts that attempt to override instructions.

Day 7 — Mini project

- Build a summariser with structured output and failure handling.
- Test empty/long input, malformed output and provider errors.

Practice/output: Deliverable: summarisation prototype with tests and README.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 3 — Spring AI + Spring Boot

Weekly goal: Integrate LLM capabilities into a Java backend using familiar enterprise patterns.

Weekly deliverable: Java REST service integrated with an LLM.

Day 1 — Setup

- Read current Spring AI docs and choose compatible versions.
- Create a Maven/Spring Boot project and configure one model provider securely.

Practice/output: Run a minimal Spring Boot model call.

Day 2 — ChatClient and service design

- Explore ChatClient and relevant abstractions for your chosen version.
- Wrap model access in a service; use constructor injection and configuration properties.

Practice/output: Implement an AI service and REST endpoint.

Day 3 — REST contracts

- Design DTOs, validation and consistent error responses.
- Separate public API contracts from provider-specific responses.

Practice/output: Create POST /api/summarize with validation.

Day 4 — Conversation context

- Compare single-turn and multi-turn interactions.
- Bound conversation history and consider privacy and retention.

Practice/output: Prototype a chat endpoint with bounded context.

Day 5 — Streaming and async

- Understand streaming, asynchronous requests, cancellation and client disconnects.
- Use async behaviour only where it improves the user experience.

Practice/output: Document or prototype a streaming response.

Day 6 — Configuration and portability

- Use environment-specific profiles and environment variables.
- Separate provider-specific code; ensure logs do not expose secrets.

Practice/output: Switch test/development configuration safely.

Day 7 — Review

- Test validation, empty output and provider failure.
- Explain controller → service → model-provider flow.

Practice/output: Deliverable: Java REST service integrated with an LLM.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 4 — Embeddings, Vector Search + RAG Foundations

Weekly goal: Understand retrieval-augmented generation and implement retrieval.

Weekly deliverable: working document retrieval component.

Day 1 — Embeddings

- Learn vector embeddings, semantic similarity and cosine similarity conceptually.
- Understand that embedding models/dimensions must be compatible.

Practice/output: Experiment with semantic search queries.

Day 2 — Ingestion and chunking

- Extract text from PDFs or text files.
- Learn chunk size, overlap, metadata and preserving headings/source information.

Practice/output: Create chunks with source metadata.

Day 3 — Vector store

- Compare vector search with traditional relational queries.
- Start with PostgreSQL/pgvector or one supported vector store.

Practice/output: Store sample embeddings and retrieve nearest chunks.

Day 4 — Retrieval quality

- Learn top-k, similarity thresholds and metadata filters.
- Compare relevant and irrelevant results across test queries.

Practice/output: Score retrieval for ten sample questions.

Day 5 — RAG architecture

- Learn ingestion → chunking → embeddings → retrieval → context → generation.
- Plan grounded answers, source references and an unknown-answer fallback.

Practice/output: Draw and explain a RAG architecture.

Day 6 — Java retrieval service

- Use Spring AI abstractions supported by your chosen version.
- Keep ingestion and query paths separate.

Practice/output: Retrieve relevant chunks from a Java service.

Day 7 — Review

- Experiment with chunk size/top-k and record retrieval errors.
- Explain RAG vs fine-tuning.

Practice/output: Deliverable: working document retrieval component.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 5 — Build an Enterprise Document Q&A; Assistant

Weekly goal: Turn retrieval into a useful portfolio project with grounded answers.

Weekly deliverable: end-to-end RAG document assistant.

Day 1 — Requirements and API design

- Define users, supported files, endpoints and unsupported-question behaviour.
- Document requirements and API contracts.

Practice/output: Create project README and architecture outline.

Day 2 — Document ingestion

- Implement file/text ingestion; extract text and preserve source/page metadata.
- Validate file type/size and handle parsing failures.

Practice/output: Ingest several safe sample documents.

Day 3 — Retrieval and generation

- Retrieve relevant chunks and pass bounded context to the model.
- Ask the model to distinguish supported facts from unknowns.

Practice/output: Answer questions grounded in the document set.

Day 4 — Citations and abstention

- Return source references mapped to actual retrieved chunks.
- Add fallback behaviour when evidence is insufficient.

Practice/output: Test answerable and unanswerable questions.

Day 5 — API resilience

- Handle empty queries, no documents and provider/vector-store failures.
- Add request IDs, validation, timeouts and consistent errors.

Practice/output: Demonstrate graceful failure cases.

Day 6 — UX and documentation

- Add a simple CLI or minimal UI if time permits.
- Document setup, environment variables, sample requests and limitations.

Practice/output: Prepare a repeatable demo script.

Day 7 — Project checkpoint

- Run ingestion-to-answer end to end.
- Review architecture and list the five highest-value improvements.

Practice/output: Deliverable: end-to-end RAG document assistant.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 6 — RAG Evaluation, Quality + Reliability

Weekly goal: Measure answer and retrieval quality rather than relying on a few demos.

Weekly deliverable: reproducible RAG evaluation suite.

Day 1 — Evaluation dataset

- Create 20–30 questions with expected sources and answer points.
- Include answerable, ambiguous and unanswerable questions.

Practice/output: Build a repeatable evaluation dataset.

Day 2 — Retrieval metrics

- Learn recall@k and precision concepts.
- Compare chunking, top-k and metadata filters.

Practice/output: Record retrieval results for your test set.

Day 3 — Answer evaluation

- Assess correctness, relevance, completeness and grounding.
- Use human review as a baseline; treat model-based grading cautiously.

Practice/output: Create an answer-quality rubric.

Day 4 — Hallucination tests

- Test missing facts, conflicting sources and unsupported questions.
- Verify citations against retrieved chunks.

Practice/output: Add regression tests for unsupported answers.

Day 5 — Improve systematically

- Change one variable at a time: prompt, chunking, top-k or filters.
- Compare against the same dataset.

Practice/output: Document before/after results.

Day 6 — Performance and resilience

- Measure latency by stage; test model/vector-store timeouts.
- Consider caching only when privacy and freshness permit.

Practice/output: Write a performance/failure report.

Day 7 — Review

- Explain how to evaluate RAG in production.
- Document known quality gaps and evaluation steps.

Practice/output: Deliverable: reproducible RAG evaluation suite.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 7 — Tool Calling + Java Service Integration

Weekly goal: Let the model request controlled functions while Java enforces business rules and access.

Weekly deliverable: controlled tool-calling assistant.

Day 1 — Tool calling

- Learn tool schemas, arguments and model-generated tool requests.
- Distinguish a tool request from execution; validate all arguments.

Practice/output: Define two read-only tools for a support assistant.

Day 2 — Implement tools

- Create Java service methods with DTOs and validation.
- Avoid arbitrary SQL/URLs and return only necessary data.

Practice/output: Implement mock order-status and knowledge-search tools.

Day 3 — Connect tools

- Register tools using the API supported by your Spring AI version.
- Handle invalid calls, tool errors and unavailable services.

Practice/output: Demonstrate a model-triggered read-only tool call.

Day 4 — Authorization

- Use trusted authenticated identity, not model-generated identity.
- Enforce tenant/resource access in backend services.

Practice/output: Test that users cannot access another user's mock order.

Day 5 — Bounded workflows

- Limit steps, deadlines and retries.
- Prevent endless loops and repeated side effects.

Practice/output: Build a bounded workflow using two tools.

Day 6 — Audit and confirmation

- Log tool name, request ID, outcome and timing without secrets.
- Require confirmation for consequential writes; consider idempotency.

Practice/output: Add a simulated action requiring confirmation.

Day 7 — Review

- Explain LLM vs tool calling vs agent.
- Test malicious/malformed arguments and document trust boundaries.

Practice/output: Deliverable: controlled tool-calling assistant.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 8 — Agentic Workflows + Orchestration

Weekly goal: Know when agents help and how to constrain multi-step behaviour.

Weekly deliverable: bounded, observable multi-step workflow.

Day 1 — Agent vs workflow

- Compare fixed orchestration with model-directed steps.
- Learn state, tools, planning loop and stop conditions.

Practice/output: Draw a workflow and identify decisions that truly need an LLM.

Day 2 — State and context

- Define state that persists between steps.
- Keep user/session boundaries explicit and handle stale context.

Practice/output: Implement an explicit workflow state object.

Day 3 — Tool selection and limits

- Restrict available tools and describe them clearly.
- Set step, time, token and cost limits.

Practice/output: Add step counter and safe termination.

Day 4 — Human approval

- Separate proposed actions from execution.
- Make approval state explicit and auditable.

Practice/output: Create a ticket-creation proposal that waits for approval.

Day 5 — Orchestration concepts

- Explore Spring AI workflow features or LangGraph concepts only as needed.
- Understand state transitions, checkpoints and retries.

Practice/output: Compare plain Java orchestration with an agent approach.

Day 6 — Failure/security scenarios

- Test prompt injection, tool failures and contradictory results.
- Prevent unsafe chains, unbounded retries and credential exposure.

Practice/output: Create an adversarial test checklist.

Day 7 — Review

- Explain where autonomy helps and where deterministic code is safer.
- Measure completion/success rather than judging demos only.

Practice/output: Deliverable: bounded, observable multi-step workflow.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 9 — Enterprise Security + Data Protection

Weekly goal: Apply backend security practices to AI endpoints and retrieval.

Weekly deliverable: documented, tested security controls.

Day 1 — Authentication and authorization

- Review Spring Security and trusted identity propagation.
- Apply least privilege to AI-connected services.

Practice/output: Protect endpoints and test unauthenticated requests.

Day 2 — Tenant-aware retrieval

- Understand document-level and tenant-level access controls.
- Apply filters before context reaches the model.

Practice/output: Add tenant metadata and cross-tenant access tests.

Day 3 — Prompt injection

- Treat documents, user prompts and tool output as untrusted.
- Use allow-listed tools and backend policy checks.

Practice/output: Test adversarial documents and blocked actions.

Day 4 — Privacy and secrets

- Use environment variables/secrets management; minimise sensitive data sent to providers.
- Review logs and retention.

Practice/output: Audit code and logs for secrets or sensitive content.

Day 5 — Output validation

- Validate model output before business use.
- Never execute arbitrary model-generated SQL, shell commands or code.

Practice/output: Add tests for malformed and unexpected output.

Day 6 — Governance and audit

- Record request IDs, prompt/model versions where feasible and outcomes.
- Document intended use, limitations and human review.

Practice/output: Add security/privacy documentation.

Day 7 — Security review

- Threat-model cross-user access, injection, secret exposure and error leakage.
- Prioritise fixes by risk.

Practice/output: Deliverable: documented, tested security controls.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 10 — Testing, Observability + Production Readiness

Weekly goal: Make the application maintainable, measurable and diagnosable.

Weekly deliverable: testable, observable and resilient service.

Day 1 — Unit testing

- Use JUnit 5 and Mockito for service logic.
- Mock model responses; test validation and error mapping.

Practice/output: Add unit tests for core service and failure paths.

Day 2 — Integration testing

- Test REST endpoints, persistence and vector-store integration.
- Use test containers or isolated services where practical.

Practice/output: Create repeatable integration tests.

Day 3 — AI regression testing

- Use a fixed dataset and rubric for answer quality.
- Test grounding, abstention and structured-output validity.

Practice/output: Create a manual release evaluation step.

Day 4 — Observability

- Add request IDs and timing for retrieval/model calls.
- Track errors, latency and token usage where available; avoid logging secrets.

Practice/output: Trace a failed request from logs.

Day 5 — Resilience

- Use timeouts, bounded retries and concurrency limits.
- Understand circuit breakers/bulkheads and safe fallbacks.

Practice/output: Simulate provider outage and show safe failure.

Day 6 — Performance and cost

- Measure latency by stage; review token limits, prompt size and model choice.
- Check parallelism and caching trade-offs.

Practice/output: Write a cost/latency optimisation plan.

Day 7 — Review

- Review configuration, tests, logs, secrets and runbook steps.
- Explain how the service would be monitored.

Practice/output: Deliverable: testable, observable and resilient service.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 11 — Docker, CI/CD + Cloud Deployment

Weekly goal: Package the app reproducibly and understand secure deployment.

Weekly deliverable: containerised app with build/test/deployment instructions.

Day 1 — Docker

- Learn images, containers, ports, volumes, environment variables and networks.
- Create a Dockerfile; keep secrets out of image layers.

Practice/output: Build and run the app in a container.

Day 2 — Local stack

- Use Docker Compose for app/dependencies if useful.
- Configure persistence, health checks and startup/shutdown.

Practice/output: Start the stack reproducibly.

Day 3 — CI basics

- Understand build, unit test, integration test and artifact stages.
- Use an approved CI platform and its secret store.

Practice/output: Create a build/test pipeline.

Day 4 — Cloud architecture

- Choose one cloud based on target jobs: Azure, AWS or GCP.
- Understand managed hosting, identity, secrets, networking and monitoring.

Practice/output: Draw a secure cloud deployment architecture.

Day 5 — Deployment

- Deploy only with access and budget approval; otherwise document steps.
- Use environment config, health checks and restricted access.

Practice/output: Create deployment guide or running demo.

Day 6 — Operations

- Review logs, health endpoints, alerts, quotas and cost budgets.
- Write troubleshooting steps for latency or provider failures.

Practice/output: Create a basic operational runbook.

Day 7 — Deployment checkpoint

- Run smoke tests; verify configuration and secret handling.
- Update setup instructions and architecture diagram.

Practice/output: Deliverable: containerised app with build/test/deployment instructions.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Week 12 — Portfolio, Interviews + Applications

Weekly goal: Turn your learning into a clear portfolio and prepare to explain design decisions.

Weekly deliverable: portfolio project, interview notes and targeted application plan.

Day 1 — Polish project

- Fix highest-impact bugs and incomplete features.
- Use safe sample data and make setup reproducible.

Practice/output: Create a stable demo version.

Day 2 — README and architecture

- Document problem, user, architecture, APIs, setup, tests and limitations.
- Explain security and failure handling.

Practice/output: Publish professional project documentation.

Day 3 — Java/Spring refresh

- Revise collections, concurrency, streams, JVM, Spring Boot, REST and transactions.
- Practise coding problems and explain trade-offs.

Practice/output: Prepare concise senior-level answers.

Day 4 — GenAI interview practice

- Explain LLMs, embeddings, RAG, chunking, vector search and tool calling.
- Discuss RAG vs fine-tuning, injection, evaluation and hallucination controls.

Practice/output: Answer 15–20 questions aloud.

Day 5 — System design

- Design multi-tenant document Q&A; with auth and audit logs.
- Discuss scaling, caching, latency, vector-store performance, cost and outages.

Practice/output: Complete one 30–45 minute design exercise.

Day 6 — Mock interview

- Give a 5–10 minute project walkthrough.
- Practise follow-up questions on alternatives, failures and lessons learned.

Practice/output: Record gaps and improve answers.

Day 7 — Job search

- Search Java GenAI Developer, GenAI Engineer, RAG, Spring AI and AI Application Engineer roles.
- Compare job descriptions and tailor resume claims to real work.

Practice/output: Deliverable: portfolio project, interview notes and targeted application plan.

End-of-week self-check: Can you explain the key concepts without notes, demonstrate the deliverable, and describe at least one limitation or failure mode?

Interview Preparation Checklist

LLM fundamentals

- ■ What are tokens, context windows, temperature and hallucinations?
- ■ How are prompting, RAG and fine-tuning different?
- ■ How do you validate structured model output?
- ■ How do you handle rate limits, timeouts and provider failures?

RAG and retrieval

- ■ Explain ingestion, chunking, embeddings, retrieval, generation and citations.
- ■ What are embeddings and similarity search?
- ■ How do chunk size, overlap, top-k and filters affect retrieval?
- ■ How do you evaluate retrieval and answer correctness?
- ■ What should happen when evidence is missing?

Java + Spring AI

- ■ Explain the controller → service → model-provider flow.
- ■ How do you keep provider-specific code out of business logic?
- ■ How do you test nondeterministic model responses?
- ■ How do you handle streaming, async calls, retries and timeouts?

Agents and security

- ■ What is tool calling, and how is it different from an agent?
- ■ How do you prevent unauthorised tool execution?
- ■ What are prompt injection and indirect prompt injection?
- ■ Why treat retrieved text and model output as untrusted?
- ■ When should a human approve an AI action?

Production and system design

- ■ Design a multi-tenant document Q&A service.
- ■ How would you monitor latency, token usage, errors and cost?
- ■ How would you protect confidential documents and tenant boundaries?
- ■ How would you respond to poor retrieval, high latency or provider outage?
- ■ How would you test, deploy and operate the service?

Portfolio Quality Checklist

- ■ Clear problem statement and target user.
- ■ Architecture diagram and end-to-end request flow.
- ■ Setup instructions and safe sample data.
- ■ API examples and configuration guidance.
- ■ Unit/integration tests and a small AI evaluation dataset.
- ■ Timeouts, error handling, validation and security controls.
- ■ Known limitations, cost/latency considerations and future improvements.
- ■ A concise 5–10 minute demo and clear design-trade-off explanations.

Suggested job-search keywords

Java GenAI Developer • Generative AI Engineer • AI Application Engineer • Spring AI • RAG Engineer • LLM Application Developer • AI Integration Engineer. Compare real job descriptions and prioritise recurring requirements.

Final advice

Your strongest positioning is: “I can design, build, test, secure and operate an enterprise Java application that uses GenAI responsibly.” Keep core Java, Spring Boot, microservices and system-design interview practice running alongside this roadmap.